

Отчёт по лабораторной работе №12

**Программирование в командном процессоре ОС UNIX.
Командные файлы**

Юсупова Амина Руслановна

Содержание

1	Цель работы	5
2	Теоретические сведения	6
2.1	Командные процессоры (оболочки)	6
3	Выполнение лабораторной работы	7
3.1	Задание 1	7
3.2	Задание 2	8
3.3	Задание 3	10
3.4	Задание 4	12
4	Выводы	16
5	Ответы на контрольные вопросы	17
5.1	Основное назначение	25
5.2	Синтаксис	25
5.3	Основные метасимволы bash	26
5.4	Примеры использования	27
5.5	Способы экранирования	28
5.6	Примеры	29
5.7	Примеры использования	33

Список иллюстраций

3.1	Скрипт	7
3.2	Исполняемый файл	8
3.3	Запуск	8
3.4	Скрипт	9
3.5	Исполняемый файл	9
3.6	Запуск	10
3.7	Скрипт	11
3.8	Исполняемый файл	11
3.9	Запуск	12
3.10	Скрипт	13
3.11	Исполняемый файл	13
3.12	Запуск	14
3.13	Запуск	15

Список таблиц

1 Цель работы

Изучить основы программирования в оболочке ОС UNIX/Linux. Научиться писать небольшие командные файлы

2 Теоретические сведения

2.1 Командные процессоры (оболочки)

Командный процессор (shell) — программа, обеспечивающая взаимодействие пользователя с операционной системой. В UNIX/Linux наиболее распространены следующие оболочки:

Оболочка	Описание
sh (Bourne shell)	Стандартная оболочка с базовым набором функций
csh (C-shell)	C-подобный синтаксис, история команд
ksh (Korn shell)	Сочетает свойства csh и sh
bash (Bourne Again Shell)	Наиболее популярная, совмещает все возможности

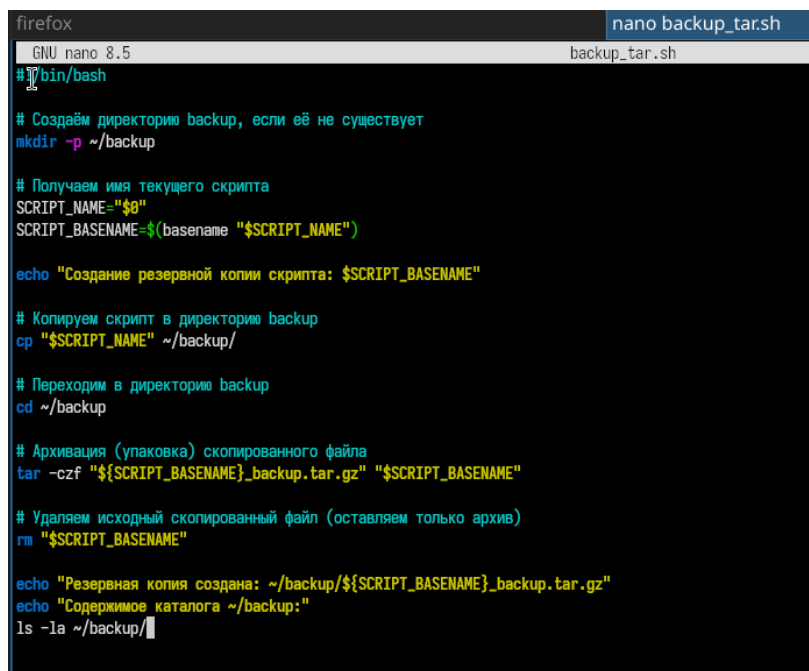
POSIX — набор стандартов для совместимости UNIX-подобных систем.

3 Выполнение лабораторной работы

3.1 Задание 1

3.1.1 1. Создание рабочего файла и написание кода

Для начала я создала рабочий файл `backup_tar.sh`. А далее я написала скрипт, который при запуске будет делать резервную копию самого себя



```
firefox nano backup_tar.sh
GNU nano 8.5 backup_tar.sh
#!/bin/bash

# Создаём директорию backup, если её не существует
mkdir -p ~/backup

# Получаем имя текущего скрипта
SCRIPT_NAME="$0"
SCRIPT_BASENAME=$(basename "$SCRIPT_NAME")

echo "Создание резервной копии скрипта: $SCRIPT_BASENAME"

# Копируем скрипт в директорию backup
cp "$SCRIPT_NAME" ~/backup/

# Переходим в директорию backup
cd ~/backup

# Архивация (упаковка) скопированного файла
tar -czf "${SCRIPT_BASENAME}_backup.tar.gz" "$SCRIPT_BASENAME"

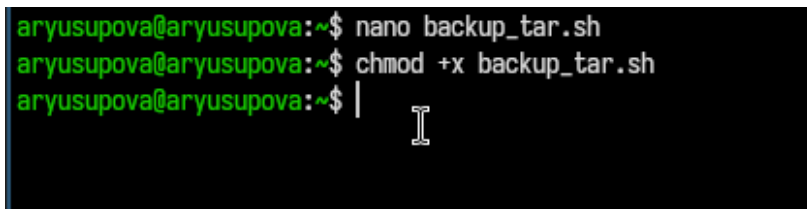
# Удаляем исходный скопированный файл (оставляем только архив)
rm "$SCRIPT_BASENAME"

echo "Резервная копия создана: ~/backup/${SCRIPT_BASENAME}_backup.tar.gz"
echo "Содержимое каталога ~/backup:"
ls -la ~/backup/
```

Рисунок 3.1: Скрипт

3.1.2 2. Исполняемый скрипт

Теперь я делаю скрипт исполняемым, чтобы программа работала

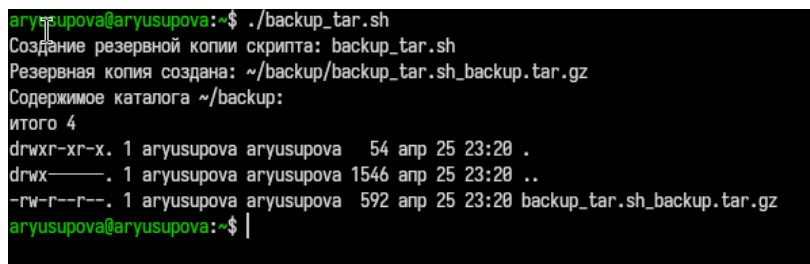


```
aryusupova@aryusupova:~$ nano backup_tar.sh
aryusupova@aryusupova:~$ chmod +x backup_tar.sh
aryusupova@aryusupova:~$ |
```

Рисунок 3.2: Исполняемый файл

3.1.3 3. Запуск скрипта

Теперь я запускаю скрипт в командной строке



```
aryusupova@aryusupova:~$ ./backup_tar.sh
Создание резервной копии скрипта: backup_tar.sh
Резервная копия создана: ~/backup/backup_tar.sh_backup.tar.gz
Содержимое каталога ~/backup:
итого 4
drwxr-xr-x. 1 aryusupova aryusupova  54 апр 25 23:20 .
drwx-----. 1 aryusupova aryusupova 1546 апр 25 23:20 ..
-rw-r--r--. 1 aryusupova aryusupova  592 апр 25 23:20 backup_tar.sh_backup.tar.gz
aryusupova@aryusupova:~$ |
```

Рисунок 3.3: Запуск

Как видно на картинке создана копия в каталоге backup

3.2 Задание 2

3.2.1 1. Создание рабочего файла и написание кода

Для начала я создала рабочий файл `print_args.sh`. А далее я написала скрипт, который обрабатывает любое произвольное число аргументов командной строки, в том числе превышающее десять. Я написала скрипт может последовательно распечатывать значения всех переданных аргументов и считать их количество.



```
firefox nano print_args.sh
GNU nano 8.5 print_args.sh
echo ""

echo "СПОСОБ 2: Перебор аргументов с помощью while и shift"

# Создаём временную копию аргументов
args_copy=("$@")
counter=1

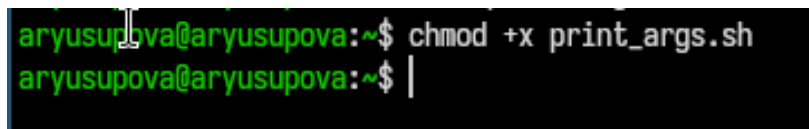
while [ $# -gt 0 ]
do
    echo " Аргумент $counter: $1"
    shift # удаляем первый аргумент, остальные сдвигаются
    counter=$((counter + 1))
done

echo ""
echo "====="
```

Рисунок 3.4: Скрипт

3.2.2 2. Исполняемый скрипт

Теперь я делаю скрипт исполняемым, чтобы программа работала



```
argusupova@argusupova:~$ chmod +x print_args.sh
argusupova@argusupova:~$
```

Рисунок 3.5: Исполняемый файл

3.2.3 3. Запуск скрипта

Теперь я запускаю скрипт в командной строке

```

argyusupova@argyusupova:~$ ./print_args.sh hello
=====
ОБРАБОТКА АРГУМЕНТОВ КОМАНДНОЙ СТРОКИ
=====
Всего передано аргументов: 1
Имя скрипта: ./print_args.sh
Все аргументы: hello

СПОСОБ 1: Перебор аргументов с помощью for
Аргумент 1: hello

СПОСОБ 2: Перебор аргументов с помощью while и shift
Аргумент 1: hello

=====
argyusupova@argyusupova:~$ ./print_args.sh 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
=====
ОБРАБОТКА АРГУМЕНТОВ КОМАНДНОЙ СТРОКИ
=====
Всего передано аргументов: 15
Имя скрипта: ./print_args.sh
Все аргументы: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

СПОСОБ 1: Перебор аргументов с помощью for
Аргумент 1: 1
Аргумент 2: 2
Аргумент 3: 3
Аргумент 4: 4
Аргумент 5: 5
Аргумент 6: 6
Аргумент 7: 7
Аргумент 8: 8
Аргумент 9: 9
Аргумент 10: 10

```

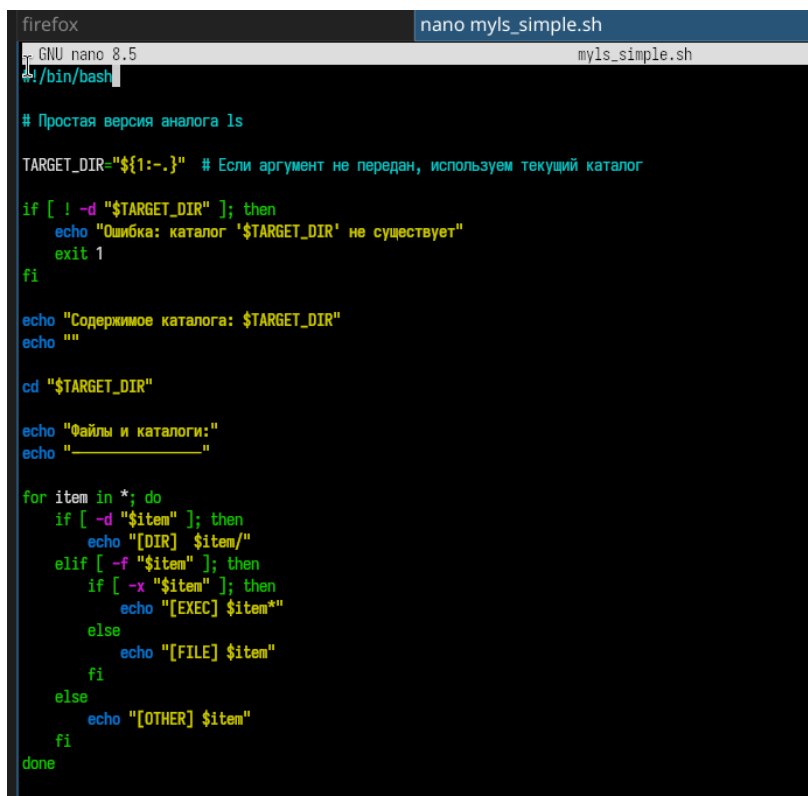
Рисунок 3.6: Запуск

Как видно на картинке программа считает количество переданных аргументов и выводит эти аргументы

3.3 Задание 3

3.3.1 1. Создание рабочего файла и написание кода

Для начала я создала рабочий файл `mysl_simple.sh`. А далее я написала скрипт, который выдает информацию о нужном каталоге и выводит информацию о возможностях доступа к файлам этого каталога.



```
firefox nano myls_simple.sh
GNU nano 8.5 myls_simple.sh
#!/bin/bash

# Простая версия аналога ls

TARGET_DIR="${1:-.}" # Если аргумент не передан, используем текущий каталог

if [ ! -d "$TARGET_DIR" ]; then
    echo "Ошибка: каталог '$TARGET_DIR' не существует"
    exit 1
fi

echo "Содержимое каталога: $TARGET_DIR"
echo ""

cd "$TARGET_DIR"

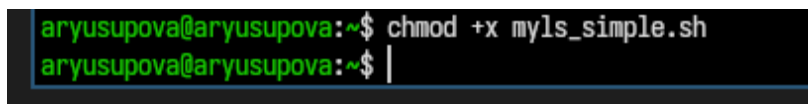
echo "Файлы и каталоги:"
echo "_____"

for item in *; do
    if [ -d "$item" ]; then
        echo "[DIR] $item/"
    elif [ -f "$item" ]; then
        if [ -x "$item" ]; then
            echo "[EXEC] $item*"
        else
            echo "[FILE] $item"
        fi
    else
        echo "[OTHER] $item"
    fi
done
```

Рисунок 3.7: Скрипт

3.3.2 2. Исполняемый скрипт

Теперь я делаю скрипт исполняемым, чтобы программа работала



```
aryusupova@aryusupova:~$ chmod +x myls_simple.sh
aryusupova@aryusupova:~$ |
```

Рисунок 3.8: Исполняемый файл

3.3.3 3. Запуск скрипта

Теперь я запускаю скрипт в командной строке

```
aryusupova@aryusupova:~$ ./mys_simple.sh
Содержимое каталога: .

Файлы и каталоги:
[FILE] 1.12.3.tar.gz
[FILE] abc1
[DIR] australia/
[DIR] backup/
[EXEC] backup_tar.sh*
[DIR] bin/
[FILE] conf.txt
[DIR] Documents/
[DIR] Downloads/
[FILE] feathers
[FILE] file1.txt
[FILE] file.txt
[DIR] git-extended/
[DIR] gitflow-avh-1.12.3/
[FILE] #lab07.sh#
[FILE] lab07.sh
[FILE] LICENSE
[FILE] may
[DIR] monthly/
[EXEC] myls_simple.sh*
[EXEC] my_os*
[DIR] play/
[EXEC] print_args.sh*
[FILE] program.c
[DIR] reports/
[DIR] second-computer/
[DIR] site/
[DIR] site-project/
```

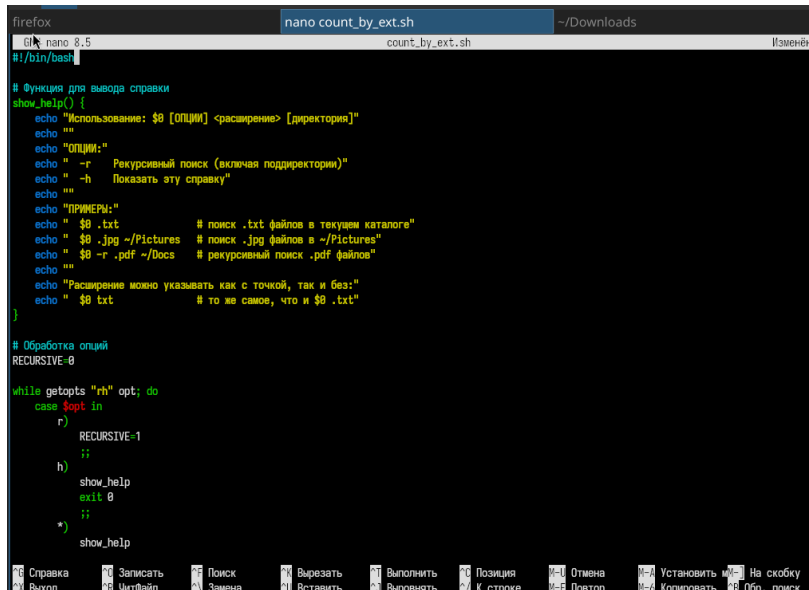
Рисунок 3.9: Запуск

Как видно на картинке программа выводит информацию о нужном каталоге.

3.4 Задание 4

3.4.1 1. Создание рабочего файла и написание кода

Для начала я создала рабочий файл `count_by_ext.sh`. А далее я написала скрипт, который получает в качестве аргумента командной строки формат файла (.txt, .doc, .jpg, .pdf и т.д.) и вычисляет количество таких файлов в указанной директории.



```
#!/bin/bash

# Функция для вывода справки
show_help() {
    echo "Использование: $0 [ОПЦИИ] <расширение> [директория]"
    echo ""
    echo "ОПЦИИ:"
    echo "  -r    Рекурсивный поиск (включая поддиректории)"
    echo "  -h    Показать эту справку"
    echo ""
    echo "ПРИМЕРЫ:"
    echo "  $0 .txt          # поиск .txt файлов в текущем каталоге"
    echo "  $0 .jpg ~/Pictures # поиск .jpg файлов в ~/Pictures"
    echo "  $0 -r .pdf ~/Docs  # рекурсивный поиск .pdf файлов"
    echo ""
    echo "Расширение можно указывать как с точкой, так и без:"
    echo "  $0 txt           # то же самое, что и $0 .txt"
}

# Обработка опций
RECURSIVE=0

while getopts "rh" opt; do
    case $opt in
        r)
            RECURSIVE=1
            ;;
        h)
            show_help
            exit 0
            ;;
        *)
            show_help
    esac
done
```

Рисунок 3.10: Скрипт

3.4.2 2. Исполняемый скрипт

Теперь я делаю скрипт исполняемым, чтобы программа работала



```
argusupova@argusupova:~$ chmod +x count_by_ext.sh
```

Рисунок 3.11: Исполняемый файл

3.4.3 3. Запуск скрипта

Теперь я запускаю скрипт в командной строке

```
aryusupova@aryusupova:~$ touch file1.txt
aryusupova@aryusupova:~$ touch file2.txt
aryusupova@aryusupova:~$ touch file3.txt
aryusupova@aryusupova:~$ touch image.jpg
aryusupova@aryusupova:~$ mkdir -p subdir
aryusupova@aryusupova:~$ cd subdir
aryusupova@aryusupova:~/subdir$ touch doc1.txt
aryusupova@aryusupova:~/subdir$ touch doc2.txt
aryusupova@aryusupova:~/subdir$ cd ..
```

Рисунок 3.12: Запуск

Как видно на картинке программа выводит количество файлов с заданным расширением.

```
aryusupova@aryusupova:~$ ./count_by_ext.sh txt
```

ПОДСЧЁТ ФАЙЛОВ ПО РАСШИРЕНИЮ

Директория: .
Расширение: .txt
Режим: только текущая директория

Найденные файлы:

./file1.txt
./file.txt
./conf.txt
./text.txt
./file2.txt
./file3.txt

ИТОГО: 6 файлов с расширением .txt

```
aryusupova@aryusupova:~$ ./count_by_ext.sh jpg
```

ПОДСЧЁТ ФАЙЛОВ ПО РАСШИРЕНИЮ

Директория: .
Расширение: .jpg
Режим: только текущая директория

Найденные файлы:

./image.jpg

I

ИТОГО: 1 файлов с расширением .jpg

Рисунок 3.13: Запуск

4 Выводы

В ходе выполнения 12 лабораторной работы я познакомилась с основами программирования в командной оболочке `bash` и получила практические навыки создания скриптов для автоматизации задач в ОС Linux.

5 Ответы на контрольные вопросы

5.0.1 1. Объясните понятие командной оболочки. Приведите примеры командных оболочек. Чем они отличаются?

Командная оболочка (shell) — это программа, которая обеспечивает интерфейс между пользователем и операционной системой, интерпретируя команды пользователя и управляя их выполнением.

Примеры командных оболочек: - **sh** (Bourne shell) — стандартная оболочка UNIX/Linux - **csh** (C-shell) — C-подобный синтаксис, история команд - **ksh** (Korn shell) — сочетает свойства csh и sh - **bash** (Bourne Again Shell) — наиболее популярная, расширенная версия sh

Отличия: оболочки отличаются синтаксисом, набором встроенных команд, возможностями настройки и дополнительными функциями (история, автодополнение, поддержка массивов).

5.0.2 2. Что такое POSIX?

POSIX (Portable Operating System Interface for Computer Environments) — набор стандартов IEEE, описывающих интерфейсы между операционной системой и прикладными программами. Обеспечивает совместимость различных UNIX/Linux-подобных систем и переносимость программ на уровне исходного кода.

5.0.3 3. Как определяются переменные и массивы в языке программирования bash?

Переменные

В bash переменные имеют тип «строка символов». Имя переменной может быть выбрано пользователем. Присвоение значения выполняется следующим образом:

```
mark=/usr/andy/bin  
name="John"  
count=10
```

Для получения значения переменной используется метасимвол \$:

```
echo $name  
mv afile ${mark}
```

Чтобы имя переменной не сливалось с последующими символами, используется запись \${имя}:

```
b=/tmp/andy-  
ls -l myfile > ${b}ls
```

Массивы

Для создания массива используется команда set с флагом -A:

```
set -A states Delaware Michigan "New Jersey"
```

За флагом следует имя переменной, а затем список значений, разделённых пробелами. Индексация массивов начинается с нулевого элемента.

Добавление элемента в массив:

```
states[49]=Alaska
```

Обращение к элементу массива:

```
echo ${states[0]}      # Delaware
echo ${states[49]}     # Alaska
echo ${states[*]}      # все элементы массива
```

5.0.4 4. Каково назначение операторов let и read?

Оператор let

Команда `let` используется для выполнения арифметических вычислений в `bash`. Она показывает, что последующие аргументы представляют собой выражение, подлежащее вычислению.

Основные особенности `let`: - Для идентификации переменной ей не нужен знак `$` - Поддерживает целочисленные арифметические операции - Результат вычисления может быть присвоен переменной

Примеры использования:

```
let sum=x+7           # сложение
let diff=x-7          # вычитание
let prod=x*7          # умножение
let quot=x/7          # целочисленное деление
let rem=x%7           # остаток от деления
let x+=7              # присваивание со сложением
let x-=7              # присваивание с вычитанием
let x++              # инкремент
let x--              # декремент
```

Альтернативная форма (двойные круглые скобки):

```
(( sum = x + 7 ))  
(( x -= 1 ))  
(( y++ ))
```

Оператор **read**

Команда `read` позволяет читать значения переменных со стандартного ввода (клавиатуры). Она приостанавливает выполнение скрипта и ждёт, пока пользователь введёт данные и нажмёт Enter.

Синтаксис:

```
read переменная1 переменная2 ... переменнаяN
```

- Примеры использования:

1. Чтение одной переменной:

```
echo "Как вас зовут?"  
read name  
echo "Здравствуйте, $name!"
```

2. Чтение нескольких переменных:

```
echo "Введите день и месяц рождения:"  
read day month  
echo "Вы родились $day числа, в месяце $month"
```

3. Чтение с дополнительной переменной для «мусора»:

```
echo "Please enter Month and Day of Birth ?"  
read mon day trash
```

В этом примере:

- `mon` — получит первое слово
- `day` — получит второе слово
- `trash` — получит всё остальное (избыточно введённую информацию)

Полезные опции `read`:

- Опция Назначение `-p «текст»` - Вывести приглашение перед вводом `-t` секунды - Таймаут ожидания ввода `-s` Скрытый - ввод (например, для паролей) `-n N` - Прочитать `N` символов

Примеры с опциями:

```
# С приглашением
read -p "Введите имя: " name

# Скрытый ввод (пароль)
read -s -p "Введите пароль: " password

# С таймаутом 5 секунд
read -t 5 -p "У вас 5 секунд: " answer
```

5.0.5 5. Какие арифметические операции можно применять в языке программирования `bash`?

В языке программирования `bash` можно применять следующие арифметические операции:

Оператор	Описание	Пример
<code>+</code>	Сложение	<code>let sum=x+7</code>

Оператор	Описание	Пример
-	Вычитание	let diff=x-7
*	Умножение	let prod=x*7
/	Целочисленное деление	let quot=x/7
%	Остаток от деления (mod)	let rem=x%7
**	Возведение в степень	let pow=x**2
+=	Присваивание со сложением	let x+=7
-=	Присваивание с вычитанием	let x-=7
=	Присваивание с умножением	let x=7
/=	Присваивание с делением	let x/=7
%=	Присваивание с остатком	let x%=7
++	Инкремент (увеличение на 1)	let x++ или ((x++))
--	Декремент (уменьшение на 1)	let x-- или ((x--))

Операторы сравнения (для использования в (())):

Оператор	Описание	Пример
==	Равно	((x == y))
!=	Не равно	((x != y))
>	Больше	((x > y))
<	Меньше	((x < y))
>=	Больше или равно	((x >= y))
<=	Меньше или равно	((x <= y))

Побитовые операторы:

Оператор	Описание	Пример
&	Побитовое И	((x & y))
\	Побитовое ИЛИ	((x \ y))
^	Побитовое исключающее ИЛИ (XOR)	((x ^ y))
~	Побитовое дополнение (инверсия)	((~x))
«	Сдвиг влево	((x « 2))
»	Сдвиг вправо	((x » 2))

Логические операторы:

Оператор	Описание	Пример
&&	Логическое И	((x > 0 && y < 10))
\ \	Логическое ИЛИ	((x == 0 \ \ y == 0))
!	Логическое НЕ	((! x))

Примеры использования:

```
#!/bin/bash

x=10
y=3

# Базовые операции
let sum=x+y          # sum = 13
let diff=x-y         # diff = 7
```

```

let prod=x*y          # prod = 30
let quot=x/y          # quot = 3 (целочисленное деление)
let rem=x%y           # rem = 1 (остаток)

# Сокращённые операции
let x+=5              # x = 15
let y*=2              # y = 6

# Инкремент и декремент
let x++               # x = 16
let y--               # y = 5

# Возведение в степень
let pow=2**3          # pow = 8

# Сравнения (в двойных скобках)
if (( x > y )); then
    echo "$x больше $y"
fi

```

5.0.6 6. Что означает операция (())?

Двойные круглые скобки (()) — это конструкция языка bash, которая используется для выполнения **арифметических операций** и **арифметических вычислений**.

5.1 Основное назначение

`(( ))` позволяет выполнять целочисленные арифметические операции внутри скобок. Это более удобная и современная альтернатива команде `let`.

5.2 Синтаксис

```
(( выражение ))
```

5.2.1 7. Какие стандартные имена переменных Вам известны?

Переменная	Назначение
PATH	Список каталогов, где оболочка ищет выполняемые программы
HOME	Путь к домашнему каталогу пользователя
PS1	Основное приглашение командной строки (например, <code>\$</code> или <code>#</code>)
PS2	Вторичное приглашение (по умолчанию <code>></code>), используется при вводе многострочных команд
IFS	Разделители полей (пробел, табуляция, перевод строки)
MAIL	Путь к файлу почты
TERM	Тип используемого терминала
LOGNAME	Имя пользователя, под которым выполнен вход в систему

Переменная	Назначение
PWD	Текущий рабочий каталог
OLDPWD	Предыдущий рабочий каталог
RANDOM	Генерирует случайное целое число при каждом обращении
SECONDS	Количество секунд с момента запуска оболочки

5.2.2 8. Что такое метасимволы?

Метасимволы — это символы, которые имеют специальное значение для командного процессора (shell). Они используются для подстановки, группировки, перенаправления ввода-вывода и других операций. Когда shell встречает метасимвол, он интерпретирует его особым образом, а не как обычный символ.

5.3 Основные метасимволы bash

Метасимвол	Назначение	Пример
*	Любая строка (в том числе пустая)	<code>ls *.txt</code> — все .txt файлы
?	Любой один символ	<code>ls file.?</code> — file.1, file.a
[]	Диапазон символов	<code>ls [a-z]*</code> — файлы на буквы a-z
\	Экранирование (отмена спец. смысла)	<code>echo *</code> — выведет *

Метасимвол	Назначение	Пример
' '	Полное экранирование	echo '*' — выведет *
" "	Частичное экранирование	echo "\$HOME" — подставит путь
\$	Подстановка переменной	echo \$PATH
<	Перенаправление ввода	cat < file.txt
>	Перенаправление вывода	ls > list.txt
\	Конвейер (pipe)	ls \ grep txt
&	Фоновый режим	emacs &
;	Разделитель команд	cd; ls
()	Группировка команд в подстановке	(cd dir; ls)
{ }	Блок команд (в текущей оболочке)	{ ls; pwd; }
#	Комментарий	# это комментарий

5.4 Примеры использования

```
# * и ? — подстановка имён файлов
echo *.c           # выведет все файлы с расширением .c
echo file?.txt     # найдет file1.txt, fileA.txt

# [] — диапазоны
ls [0-9]*          # файлы, начинающиеся с цифры
```

```

# | — конвейер
cat file.txt | wc -l  # подсчёт строк

# > — перенаправление
echo "Hello" > output.txt

# ; — последовательное выполнение
mkdir test; cd test; touch file.txt

```

5.4.1 9. Как экранировать метасимволы?

Экранирование — это снятие специального смысла с метасимволов, чтобы командный процессор интерпретировал их как обычные символы.

5.5 Способы экранирования

Способ	Описание	Пример	Результат
\	Экранирует один следующий символ	echo *	Выведет *
'...'	Экранируют все метасимволы внутри	echo '*?'	Выведет *?
"..." (двойные кавычки)	Экранируют все, кроме \$, ', \, "	echo "\$HOME"	Подставит значение HOME

5.6 Примеры

```
# Экранирование через \
echo \*          # выведет *
echo \?          # выведет ?
echo \$HOME      # выведет $HOME (не подстановка)

# Экранирование через одинарные кавычки
echo '*?$#\|'    # выведет *?$#\|

# Экранирование через двойные кавычки
echo "Home: $HOME" # выведет Home: /home/user (подстановка переменной)
echo "Звездочка: *" # выведет Звездочка: * (* не работает)

# Комбинация
echo "Символ \$"  # выведет Символ $
```

5.6.1 10. Как создавать и запускать командные файлы?

Создание: 1. Создать файл с расширением .sh 2. Первая строка: `#!/bin/bash` (shebang) 3. Записать команды

Запуск:

```
chmod +x script.sh # сделать исполняемым
./script.sh        # запустить
```

5.6.2 11. Как определяются функции в языке программирования bash?

```
# Способ 1 (с ключевым словом function)
```

```
function имя {  
    команды  
}
```

```
# Способ 2 (с круглыми скобками)
```

```
имя() {  
    команды  
}
```

```
# Пример
```

```
clist() {  
    cd $1  
    ls  
}
```

5.6.3 12. Каким образом можно выяснить, является файл каталогом или обычным файлом?

```
if [ -d "$file" ]; then  
    echo "Это каталог"  
fi
```

```
if [ -f "$file" ]; then  
    echo "Это обычный файл"
```

fi

Другие проверки:

-r (чтение), -w (запись), -x (выполнение), -L (ссылка)

5.6.4 13. Каково назначение команд **set**, **typeset** и **unset**?

Команда	Назначение
set	Просмотр переменных окружения; создание массивов (set -A); установка параметров оболочки
typeset	Объявление переменных с типом (-i — целые); управление функциями (-f)
unset	Удаление переменных (unset var) или функций (unset -f func)

5.6.5 14. Как передаются параметры в командные файлы?

```
./script.sh param1 param2 param3
```

Переменная	Описание
\$0	Имя скрипта
\$1-9 #	Количество параметров
* @	Все параметры отдельными словами

5.6.6 15. Назовите специальные переменные языка **bash** и их назначение.

Переменная	Назначение
\$0	Имя командного файла (скрипта)
\$1, \$2, ..., \$9	Позиционные параметры (аргументы командной строки)
\$#	Количество переданных параметров
\$*	Все параметры как одна строка
\$@	Все параметры как отдельные слова (с учётом кавычек)
\$?	Код завершения последней выполненной команды (0 — успех, не 0 — ошибка)
\$\$	Уникальный идентификатор (PID) текущего процесса
\$_	PID последней фоновой команды
\$-	Значение флагов командного процессора
\${#name}	Длина строки в переменной name
\${name[n]}	Обращение к n-му элементу массива name
\${name[*]}	Все элементы массива name, разделённые пробелом
\${name[@]}	То же, что *, но с учётом пробелов внутри элементов
\${#name[*]}	Количество элементов в массиве name

Переменная	Назначение
<code>\${name:-value}</code>	Если name не определена или пуста, подставить value
<code>\${name:=value}</code>	Если name не определена или пуста, присвоить value
<code>\${name:?value}</code>	Если name не определена или пуста, вывести value как ошибку
<code>\${name:+value}</code>	Если name определена, подставить value
<code>\${name#pattern}</code>	Удалить самый короткий левый образец (pattern)
<code>\${name##pattern}</code>	Удалить самый длинный левый образец
<code>\${name%pattern}</code>	Удалить самый короткий правый образец
<code>\${name%%pattern}</code>	Удалить самый длинный правый образец
<code>\${name/pattern/replacement}</code>	Заменить первое вхождение pattern на replacement
<code>\${name//pattern/replacement}</code>	Заменить все вхождения pattern на replacement

5.7 Примеры использования

```
#!/bin/bash

# $0, $1, $#, $@
echo "Имя скрипта: $0"
echo "Первый аргумент: $1"
echo "Количество аргументов: $#"
```

echo "Все аргументы: \$@"


```
# $? — код завершения
ls /home
echo "Код завершения ls: $?"
```



```
# $$ — PID процесса
echo "PID текущего скрипта: $$"
```



```
# Длина строки
name="Hello"
echo "Длина строки: ${#name}"      # 5
```



```
# Массивы
set -A fruits apple banana cherry
echo "Первый элемент: ${fruits[0]}"
echo "Все элементы: ${fruits[*]}"
echo "Количество элементов: ${#fruits[*]}"
```



```
# Подстановка значений по умолчанию
echo "${var:-default}"      # выведет default, если var не определена
```